

MEMORIAL TÉCNICO DE DESENVOLVIMENTO

PORTAL DE SOLICITAÇÕES INTERNAS

Mini-Projeto Full Stack

Java 21 • Spring Boot 4.1.1 • Angular 17 • TypeScript • PostgreSQL
Spring Security • Swagger/OpenAPI

José Gilson de Lucena Barbosa Filho

2026

Apresentação

Este Memorial Técnico de Desenvolvimento apresenta não apenas as funcionalidades implementadas no Portal de Solicitações Internas, mas principalmente o processo decisório adotado durante o desenvolvimento. O documento registra as tecnologias selecionadas, as razões para sua escolha, as decisões arquiteturais, as estratégias de segurança e comunicação, os problemas encontrados, as limitações da solução e as melhorias que seriam consideradas em uma evolução para ambiente corporativo de produção.

O projeto foi desenvolvido como uma aplicação full stack, com frontend em Angular, backend em Java/Spring Boot e persistência em PostgreSQL, utilizando API REST para integração entre as camadas.

1. Identificação do projeto

Nome: Portal de Solicitações Internas.

Tipo: Mini-Projeto Full Stack.

Responsável: José Gilson de Lucena Barbosa Filho.

Objetivo: disponibilizar uma aplicação web para cadastro, consulta, acompanhamento, edição, alteração de status e exclusão de solicitações internas.

2. Contexto e problema

O problema proposto consiste em centralizar solicitações internas em uma aplicação que permita registrar demandas e acompanhar seu ciclo de atendimento. Sem uma solução centralizada, informações podem ficar dispersas e o acompanhamento do andamento das solicitações pode se tornar menos organizado.

A solução adotada foi um portal web com autenticação, persistência em banco relacional, API REST e interface para gerenciamento das solicitações.

3. Objetivos técnicos

- Separar claramente frontend, backend e persistência.
- Disponibilizar uma API REST para as operações do domínio.
- Implementar autenticação e controle de sessão.
- Proteger recursos que exigem usuário autenticado.
- Persistir solicitações e usuários em banco relacional.
- Disponibilizar uma interface simples para cadastro, consulta e gerenciamento.
- Documentar e testar a API durante o desenvolvimento.

4. Tecnologias utilizadas

Tecnologia/	Categoria	Uso no projeto	Justificativa técnica
-------------	-----------	----------------	-----------------------

Ferramenta			
Java 21	Linguagem	Implementação do backend	Tipagem forte, maturidade e adequação a aplicações corporativas.
Spring Boot 4.1.1	Framework backend	API REST e infraestrutura da aplicação	Reduz configuração e integra recursos do ecossistema Spring.
Spring Security	Segurança	Autenticação, sessão e CSRF	Integra segurança ao backend e evita implementação manual de mecanismos críticos.
Angular 17	Framework frontend	Interface, rotas e componentes	Estrutura o frontend em componentes e serviços e favorece manutenção.
TypeScript	Linguagem	Código do frontend	Tipagem estática e melhor organização de aplicações Angular.
PostgreSQL	Banco de dados	Persistência	Banco relacional maduro e adequado a entidades relacionadas.
Swagger/OpenAPI	Documentação/testes	Documentação e validação dos endpoints	Facilita entendimento e teste da API.
Node.js / npm	Runtime/gerenciador	Execução e dependências Angular	Ecossistema padrão para desenvolvimento do frontend Angular.
VS Code	Ferramenta	Desenvolvimento	Ambiente leve e adequado ao conjunto de tecnologias utilizado.

5. Justificativa técnica das tecnologias

5.1 Java 21

A escolha do Java 21 foi motivada pela maturidade da linguagem, tipagem forte e ampla utilização em sistemas corporativos. Para o cenário proposto, Java oferece uma base estável para construção de regras de negócio e APIs.

Em relação a linguagens mais dinâmicas, a tipagem e as ferramentas do ecossistema favorecem a identificação antecipada de inconsistências. A principal vantagem para manutenção é a organização do código e a previsibilidade das interfaces entre componentes.

Para uma solução maior, a mesma base permite evolução do domínio, integração com outros serviços e adoção de práticas corporativas sem necessidade de troca de linguagem.

5.2 Spring Boot

Spring Boot foi escolhido para reduzir a configuração necessária para uma aplicação Java web e permitir integração direta com recursos como Spring Security e persistência.

Uma alternativa seria construir uma aplicação Java web com configuração mais manual. Para o escopo proposto, isso aumentaria o esforço de infraestrutura sem trazer benefício proporcional.

A principal contribuição para produtividade é permitir que o desenvolvimento se concentre nas regras do negócio, mantendo uma estrutura organizada em controllers, services, repositories e entidades.

5.3 Spring Security

Spring Security foi utilizado para autenticação, controle de acesso, gerenciamento da sessão e proteção CSRF.

A alternativa de implementar autenticação manualmente aumentaria o risco de falhas de segurança e o custo de manutenção. A adoção de uma biblioteca consolidada reduz a necessidade de desenvolver mecanismos críticos do zero.

Para produção, a configuração poderia evoluir para perfis de acesso, políticas mais granulares, gerenciamento seguro de credenciais e integração com um provedor corporativo de identidade, conforme o cenário.

5.4 Angular e TypeScript

Angular foi escolhido para estruturar o frontend em componentes, serviços, rotas, guardas e formulários. TypeScript complementa essa escolha com tipagem estática.

Uma alternativa seria utilizar JavaScript sem framework ou outro framework frontend. Para o projeto proposto, Angular oferece uma estrutura mais padronizada para organização de uma aplicação com múltiplas telas e regras de navegação.

A separação entre componentes e serviços facilita manutenção e evolução, além de favorecer reutilização.

5.5 PostgreSQL

PostgreSQL foi escolhido porque o domínio possui entidades relacionadas, como usuários e solicitações, e se beneficia de um banco relacional.

Uma solução NoSQL poderia ser adequada a cenários com estrutura de dados menos relacional, porém não apresentaria vantagem clara para este domínio. O PostgreSQL oferece consistência, relacionamentos e recursos de consulta adequados ao problema.

A escolha também favorece uma futura evolução para índices, paginação, auditoria e consultas mais elaboradas.

5.6 Swagger/OpenAPI

Swagger foi utilizado para documentar e testar os endpoints da API durante o desenvolvimento. A ferramenta reduziu o atrito para validar requisições e respostas e permitiu verificar o comportamento da API sem depender exclusivamente da interface Angular.

Em um ambiente corporativo, a documentação OpenAPI também pode ser utilizada como referência para consumidores da API e integração entre equipes.

6. Justificativa conceitual e arquitetura

A arquitetura foi organizada em três blocos principais: frontend Angular, backend Spring Boot e banco PostgreSQL.

O frontend é responsável pela apresentação, navegação, formulários, filtros e consumo HTTP. O backend concentra autenticação, regras de negócio e acesso aos dados. O banco mantém a persistência.

A comunicação ocorre por HTTP utilizando endpoints REST. Essa separação permite que frontend e backend evoluam de forma relativamente independente.

7. Organização das camadas do backend

O backend foi estruturado seguindo uma separação de responsabilidades.

- Controller: recebe requisições HTTP e devolve respostas da API.
- Service: concentra regras de negócio e operações do domínio.
- Repository: abstrai o acesso aos dados persistidos.
- Entity: representa os objetos persistidos no banco.
- DTO: representa dados de entrada da API, reduzindo o acoplamento entre requisições e entidades.

Essa organização não foi adotada apenas por convenção: ela facilita localizar responsabilidades, testar componentes e alterar uma camada sem concentrar toda a lógica em um único arquivo.

8. Modelagem de dados

O domínio foi modelado inicialmente com as entidades Usuário e Solicitação. Uma solicitação possui título, descrição, categoria, status, data de criação e referência ao solicitante.

A relação entre usuário e solicitação permite identificar quem registrou a demanda. A escolha do modelo relacional acompanha a natureza do domínio e facilita consultas envolvendo os registros relacionados.

9. Padrões e princípios utilizados

O projeto utiliza conceitos de arquitetura em camadas, separação de responsabilidades e injeção de dependência oferecida pelo Spring.

No frontend, a organização em componentes e serviços segue a estrutura recomendada pelo Angular. O interceptor centraliza o envio de credenciais e o guard centraliza a verificação de acesso à rota protegida.

Não foi adotado um padrão arquitetural mais complexo, como microserviços, porque o escopo não justificava a complexidade operacional adicional.

10. Estratégia de autenticação e segurança

A autenticação foi implementada com Spring Security e sessão HTTP. O login é processado pelo backend e a sessão é mantida por cookie de sessão.

O frontend utiliza withCredentials para permitir o envio das credenciais de sessão nas requisições. Também foi implementada consulta ao endpoint de identificação do usuário autenticado para validar a sessão.

A proteção CSRF foi mantida nas operações protegidas. O token é obtido pelo frontend e enviado no cabeçalho correspondente quando necessário.

O CORS foi configurado para permitir a comunicação entre o frontend e o backend durante o desenvolvimento local.

11. Estratégia de comunicação frontend/backend

Angular consome a API REST por meio de serviços. A responsabilidade de comunicação foi retirada dos componentes sempre que possível, mantendo os componentes focados na interação e apresentação.

O interceptor de credenciais garante o envio da sessão. O Auth Guard consulta o backend antes de permitir acesso à área protegida.

Essa abordagem reduz duplicação e centraliza comportamentos transversais.

12. Processo decisório durante o desenvolvimento

Uma das decisões mais relevantes foi manter autenticação baseada em sessão em vez de introduzir JWT. Para o escopo do projeto, a sessão HTTP apresentou menor complexidade e atendeu diretamente ao requisito de controle de acesso.

Outra decisão foi manter uma aplicação monolítica modular em vez de microsserviços. O domínio e o volume do projeto não justificavam múltiplos serviços, infraestrutura de descoberta, observabilidade distribuída e complexidade de deploy.

Também foi priorizada uma API REST simples, com endpoints específicos para alteração de status, em vez de concentrar todas as operações em um único endpoint genérico. Isso torna o contrato mais explícito.

13. Regras de negócio

- Solicitações são criadas com status ABERTO.
- Os status implementados são ABERTO, EM_ATENDIMENTO e CONCLUIDO.
- Solicitações concluídas não podem ser editadas.
- A alteração de status utiliza endpoint específico.
- Operações protegidas exigem autenticação.
- A rota de solicitações é protegida no frontend.

14. Testes e validações

A API foi validada principalmente por meio do Swagger, verificando autenticação, cadastro, consulta, edição, alteração de status e exclusão.

No frontend foram realizados testes funcionais do fluxo de login, logout, navegação, proteção de rota, cadastro, filtros, edição e exclusão.

Também foi utilizado o endpoint `/api/auth/me` para confirmar que a autenticação estava efetivamente persistida na sessão.

15. Problemas encontrados e decisões de correção

Falha de configuração do DataSource: A conexão e as propriedades do banco foram ajustadas para permitir a inicialização do backend.

403 durante autenticação: Foi identificado bloqueio por CSRF no login. O fluxo foi ajustado para obter o token e permitir o processamento correto do endpoint de login.

Redirecionamento inesperado para /login: Foi investigada a persistência da autenticação e configurado o SecurityContextRepository baseado em sessão.

Sessão não chegando inicialmente ao backend pelo Angular: Foi implementado interceptor com `withCredentials`.

Acesso direto à rota protegida: Foi criado Auth Guard para consultar a sessão no backend antes de liberar a rota.

Operações protegidas com CSRF: O frontend passou a obter e enviar o token CSRF nos métodos que exigem proteção.

16. Análise crítica da solução

A solução atende ao escopo funcional proposto, porém possui limitações naturais de um projeto desenvolvido para avaliação e execução local.

- O projeto não contempla, no estado atual, uma infraestrutura de produção.
- Não foi implementado pipeline completo de CI/CD.
- Não há containerização obrigatória do ambiente.
- Não foi implementada observabilidade centralizada.
- Não há suíte abrangente de testes automatizados de unidade e integração.
- Não há mecanismo completo de auditoria das alterações.
- Não há controle de permissões baseado em múltiplos perfis de usuário.
- A listagem não foi projetada com paginação para grandes volumes.

Essas limitações não impedem o funcionamento da solução, mas representam pontos de evolução caso o sistema seja utilizado em escala maior.

17. Melhorias futuras

- Adicionar testes unitários e de integração automatizados ao backend.
- Adicionar testes automatizados de frontend e testes end-to-end.
- Implementar Docker e Docker Compose para padronização do ambiente.
- Criar pipeline CI/CD para build, testes e publicação.
- Adicionar paginação, ordenação e filtros mais avançados.
- Implementar perfis e permissões, como usuário, atendente e administrador.
- Adicionar trilha de auditoria para alterações de status e registros.
- Centralizar logs e adicionar métricas e monitoramento.
- Configurar backup e estratégias de recuperação do banco.
- Adicionar documentação de API mais detalhada e exemplos de consumo.

18. O que seria diferente em um ambiente corporativo de produção

Em produção, a arquitetura funcional poderia ser mantida, mas a infraestrutura e os controles seriam ampliados.

- HTTPS obrigatório e políticas de segurança de transporte.
- Segredos e credenciais fora do código-fonte, utilizando um mecanismo seguro de gerenciamento de secrets.
- Banco PostgreSQL gerenciado, com backup, monitoramento e política de recuperação.

- Containers para padronizar execução e implantação.
- Pipeline CI/CD com etapas de compilação, testes, análise e publicação.
- Testes automatizados executados a cada alteração relevante.
- Monitoramento de aplicação, banco e infraestrutura.
- Logs estruturados e centralizados.
- Controle de acesso por perfis e princípio do menor privilégio.
- Políticas de segurança, auditoria e rastreabilidade.
- Possibilidade de escalabilidade horizontal caso a carga do sistema justifique.
- Ambientes separados para desenvolvimento, homologação e produção.

A adoção dessas práticas seria avaliada conforme criticidade, volume de usuários, orçamento, requisitos de disponibilidade e políticas da organização. Não seria tecnicamente adequado introduzir toda essa complexidade em um mini-projeto apenas por antecipação.

19. Impacto das decisões em manutenção, escalabilidade e produtividade

A separação em camadas facilita manutenção porque cada responsabilidade possui um local definido. A utilização de serviços e repositórios reduz a concentração de lógica nos controllers.

A API REST permite que outros clientes sejam desenvolvidos futuramente sem alterar necessariamente as regras do backend.

A escolha de Angular e TypeScript favorece produtividade e organização do frontend, enquanto Spring Boot e Spring Security reduzem o esforço de infraestrutura e segurança.

A arquitetura monolítica modular foi uma escolha consciente para manter baixo o custo operacional. Caso a aplicação cresça, módulos de maior demanda podem ser identificados antes de qualquer decisão de divisão em serviços.

20. Resultado final

O resultado é uma aplicação full stack funcional para gerenciamento de solicitações internas, composta por frontend Angular, backend Java/Spring Boot e banco PostgreSQL.

A solução possui autenticação, controle de sessão, proteção CSRF, CORS, guard de rota, cadastro, consulta, filtros, edição, alteração de status e exclusão.

Além do resultado funcional, o processo de desenvolvimento permitiu identificar problemas de integração e segurança e tomar decisões técnicas baseadas no escopo e nas necessidades reais da aplicação.

21. Conclusão

O desenvolvimento do Portal de Solicitações Internas demonstrou a aplicação prática de conceitos de desenvolvimento full stack, arquitetura em camadas, APIs REST, persistência relacional, autenticação e segurança.

As escolhas tecnológicas foram orientadas principalmente por maturidade, produtividade, facilidade de manutenção e adequação ao escopo. As decisões arquiteturais buscaram evitar complexidade desnecessária sem impedir evolução futura.

A análise crítica demonstra que o projeto atende ao objetivo proposto, mas que uma implantação corporativa exigiria evolução em automação de testes, CI/CD, infraestrutura, observabilidade, auditoria, segurança operacional e escalabilidade.

Dessa forma, o memorial registra não apenas o que foi construído, mas também por que as decisões foram tomadas, quais alternativas foram consideradas e como a solução poderia evoluir em um cenário real de produção.